SIMATS ENGINEERING

Department of ComputerScience and Engineering

ASSESSMENT TOOL2 - CONCEPT MAPPING

Course Code:	CSA1205	**Course Title:**	Computer Architecture for Multicore Systems
CO Assessed:	CO1	**Assessment:**	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	**Total Marks:**	25
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	B.Vennela.	**Reg.No.:**	192411260y.
Department:	ECE	**Date:**	

Instruction to Students

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

→ 8-bit Micro processor
→ Introduced by intel in 1976

→ Introduced in intel by 1978.

Key Architecture

wordsize	8-bit ⟷	16 Bit
Address Bus	16-bit ⟷	20 bit
Data Bus	8-bit ⟷	16-bit
Memory segmentation	64KB ⟷	1MB
Registers	8-bit Registers (A, B, C, D, E, H, L) 16-bit PC, SP	16-bit Registers (AX, BX, CX, DX, SP BP, SI, DI segement Registers : CS, DS, SS, ES)

→ uses 0 and 1
→ Language of Computers

→ uses 0-9 and A-F
→ 1hex digit = 4bits

→ Format (example - 8085):

Opcode	Operand/Adress
↓ 1 byte	↓ 0, 1 or 2 byte

3E	3A

Opcode · Data.

Hexadecima acts a bridge between humans and manching by providing a compact.

(2)

CPU, Memory, I/O Structures.

CPU

ALU
→ performs arithmetic and logical operations.

CU
→ Fetches Instructions
→ Decodes Instructions
→ Issues control signals.

Registers
→ General purpose registers
→ Temporary data storage

Memory

cachememory
small very fast
→ stores frequently
→ Reduces access time.

Main
→ Large capacity
→ slower than Cache.

I/O devices

Output devices (Monitor, printer, Speakers)

Storage devices (HDD, [illegible])

System Bus.

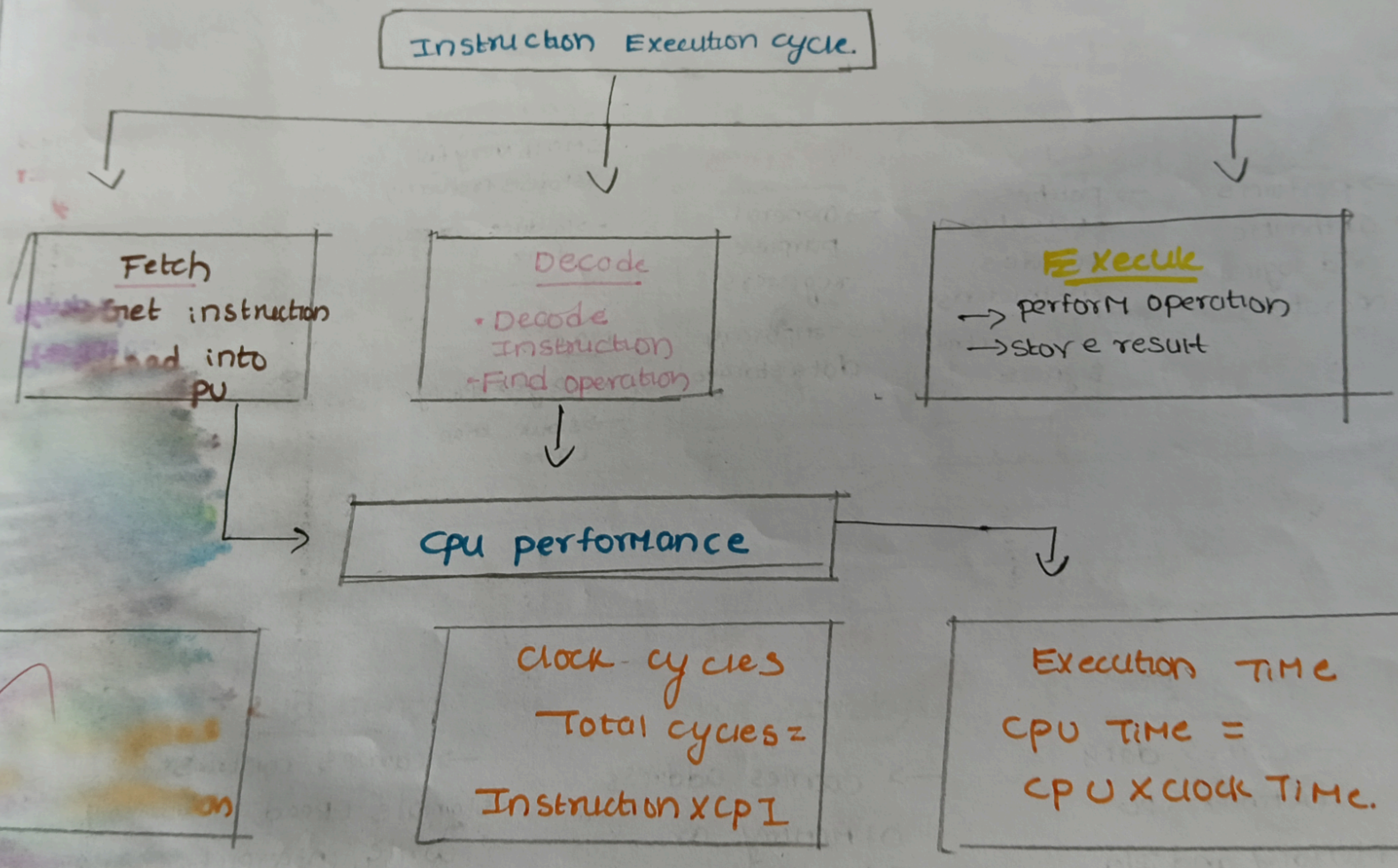
Data Bus
→ carries data between CPU Memory and I/O

Address Bus
→ carries address of Memory or I/O location

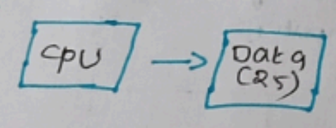
control Bus
→ carries controls signals (Read, write, interrput clock, Reset etc),

③

Instruction Execution cycle and cpu performance.

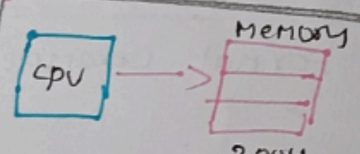


① **Immediate Addressing**
The operand is part of the Instruction.



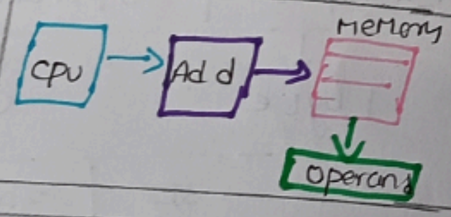
Example
MOV R1, #25

② **Direct Addressing**
The address of the operand is given in the Instruction.



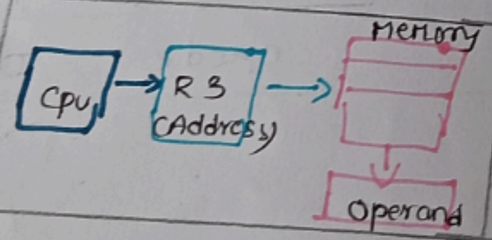
Example
MOV R1 [2000H]

③ **Indirect Addressing**
The address field points to a Memory location that contains the operand.



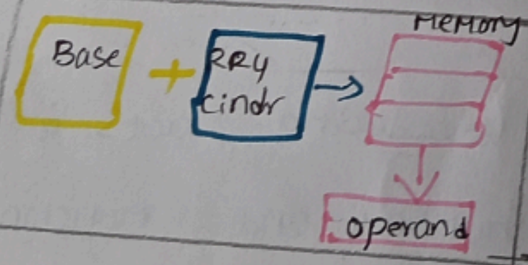
Example
MOV R1 [R2]

④ **Register indirect**
The Register contains the address of the operand.



Example
MOV R1 [R3]

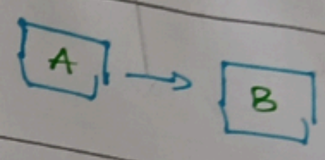
⑤ **Indexed Addressing**
The effective address is formed by adding the contents of a Register



Example
MOV R1
ARRAY [R2]

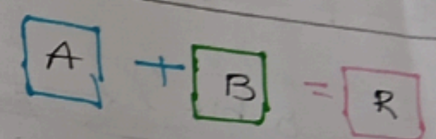
Instructions

① **Data Transfer Instructions**



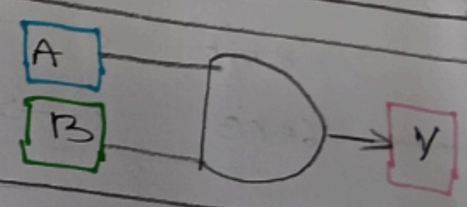
Example
MOV A,B

② **Arithmetic Instructions**



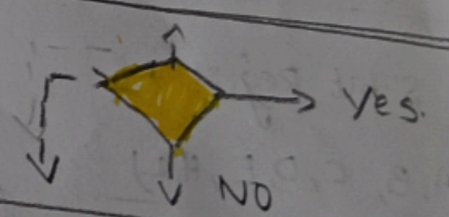
Example
ADD A,B

③ **logical Instructions**



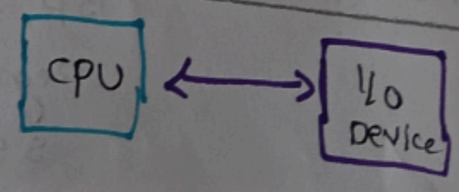
Example
AND A, B

④ **Branch/control Instructions**



Example
JMP LABEL

⑤ **Input/output Instruction**



Example
INPOR